PERFORMANCE CASE STUDY

SupportDesk

Enterprise Ticketing Platform — Performance Turnaround

React | Next.js | Debounced Search | Pagination | Large Datasets

3675ms → 2.3ms
Render Time

1500x
Faster

10,000
Records Tested

01 — OVERVIEW

Overview

SupportDesk is a React and Next.js ticketing platform for managing customer support operations — ticket management, search, advanced filtering, agent assignment, and support operations tracking. As ticket volume grew to 10,000 records, the Tickets page — the primary daily-use view for support agents — began showing significant rendering and interaction delays.

02 — THE PROBLEM

The Problem

Users reported a slow initial page load, laggy search, and slow filtering on the Tickets table. An initial React Profiler recording measured a commit render time of 3675ms on that view, with every keystroke in search and every filter change re-triggering the same expensive work.

BEFORE
3675ms

→

AFTER
2.3ms

03 — ROOT CAUSE ANALYSIS

Root Cause Analysis

- The full 10,000-ticket dataset was regenerated from scratch on every render instead of being created once and reused.
- Each filter change re-filtered, re-sorted, and re-rendered the entire ticket table rather than a scoped subset.
- Every search keystroke triggered a full dataset scan, sort, and re-render, with no memoization and additional unnecessary computation sitting directly in the search path.
- The table rendered all 10,000 rows to the DOM regardless of what was actually visible in the viewport.

04 — DIAGNOSTIC METHODOLOGY

Diagnostic Methodology

- Captured a baseline Profiler recording while typing in the search box and toggling filters, to isolate whether the bottleneck was data generation, filtering logic, or DOM rendering.
- Used the Profiler's commit-duration timeline to confirm the render cost scaled directly with dataset size rather than with the number of visible rows — a strong signal that pagination, not just memoization, was needed.
- Checked CPU usage in Chrome DevTools' Performance panel during rapid typing to quantify how much main-thread time was spent in redundant filter/sort work versus actual rendering.
- Validated each optimization incrementally — memoization first, then debouncing, then pagination — re-profiling after each change to confirm which fix contributed the most before moving to the next.

05 — OPTIMIZATIONS IMPLEMENTED

Optimizations Implemented

- Memoized dataset generation so the 10,000-ticket dataset is created once and reused across renders instead of regenerated each time.
- Memoized filtering logic so filter operations recompute only when their actual inputs (status, priority, assignee, search text) change.
- Added debounced search input alongside useDeferredValue and precomputed search text, so rapid typing no longer triggers a full scan on every keystroke.
- Removed unnecessary computation that had crept into the search path over time.
- Paginated the ticket table to 50 rows per page instead of rendering all 10,000 rows to the DOM at once.

06 — REPRESENTATIVE FIX (BEFORE)

Before Optimization

BEFORE

```
function TicketsTable({ tickets, filters }) {
  const visible = tickets
    .filter(t ⇒ matchesFilters(t, filters))
    .sort(byUpdatedDesc);
  // full 10,000-row scan + sort on every keystroke
  return visible.map(t ⇒ <TicketRow key={t.id} ticket={t} />);
}
```

06 — REPRESENTATIVE FIX (AFTER)

After Optimization

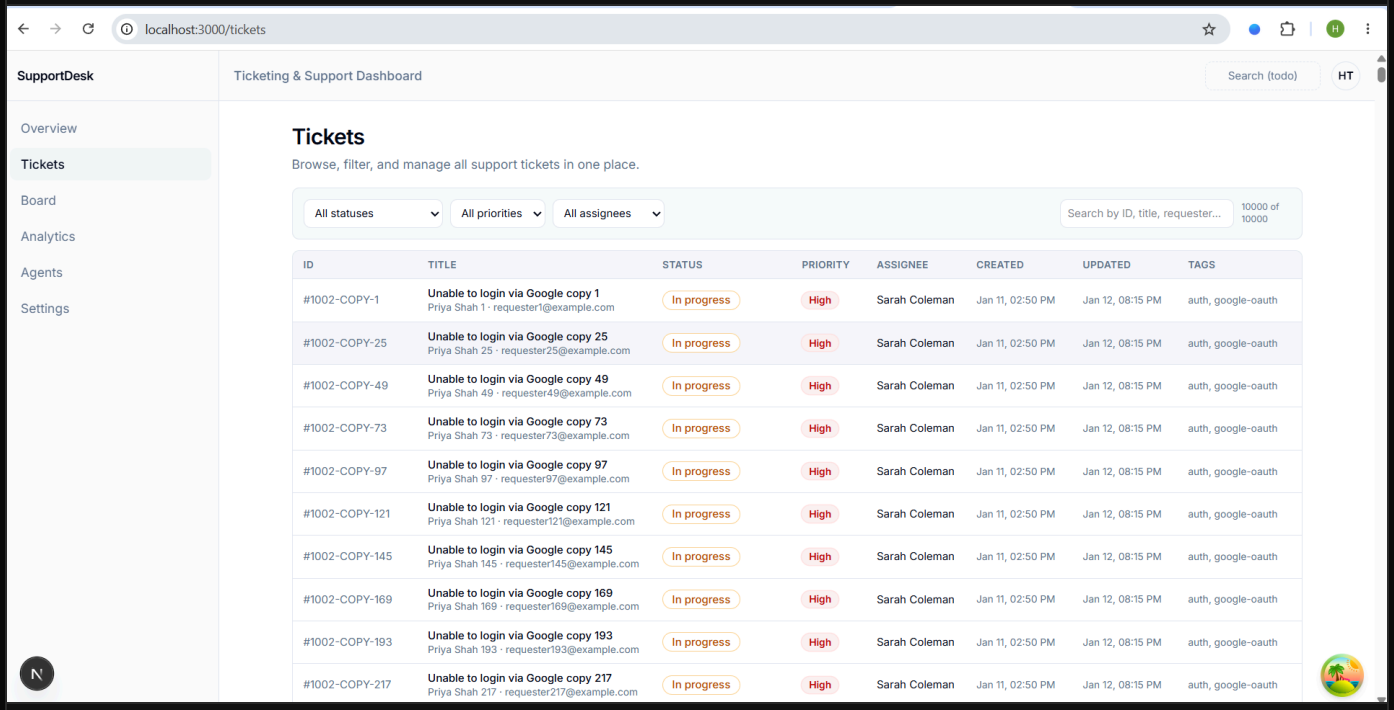
AFTER

```
function TicketsTable({ tickets, filters, page }) {
  const deferredFilters = useDeferredValue(filters);
  const visible = useMemo(
    () => tickets.filter(t => matchesFilters(t, deferredFilters))
              .sort(byUpdatedDesc),
    [tickets, deferredFilters]
  );
  const pageRows = visible.slice(page * 50, page * 50 + 50);
  return pageRows.map(t => <TicketRow key={t.id} ticket={t} />);
}
```

07 — SCREENSHOT ASSETS

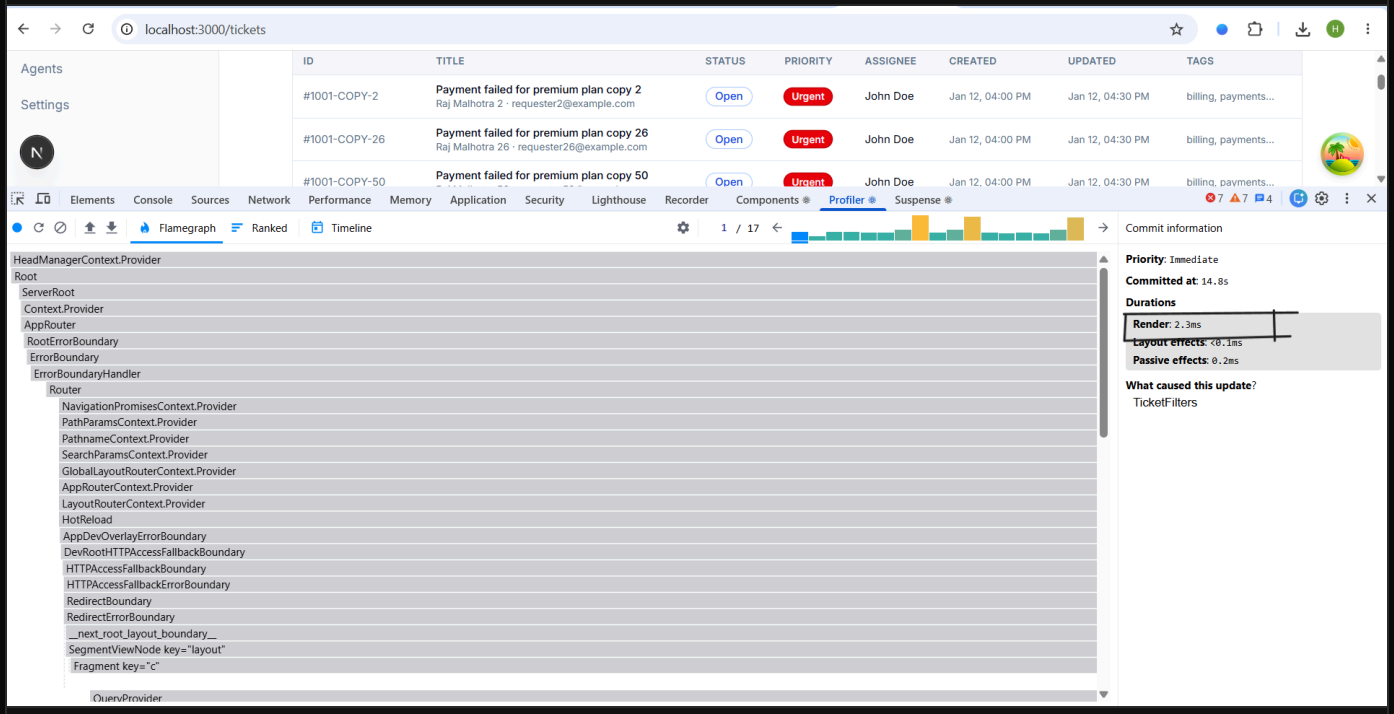
Application & Profiler Captures

SUPPORTDESK DASHBOARD



Ticketing dashboard after performance optimization.

REACT PROFILER ANALYSIS



Optimized React Profiler capture showing reduced render time.

08 — RESULTS TABLE

Results

METRIC	BEFORE	AFTER
Render Time	3675ms	2.3ms
Search Responsiveness	Laggy	Smooth
Filter Response	Slow	Fast
Rows Rendered per Page	10,000	50
CPU Usage While Typing	High	Lower
Overall Responsiveness	Poor	Strong

09 — BUSINESS IMPACT

Business Impact

Support agents work inside this ticket table for most of their shift — every second of lag on search or filtering is a second multiplied across every agent, every ticket, every day. A near-instant table (2.3ms) instead of a multi-second freeze (3675ms) turns a tool agents have to work around into one that keeps pace with how quickly they actually need to triage and respond.

10 — TECHNIQUES USED

Techniques

React Profiler · useMemo · useDeferredValue · Debounced Search · Pagination · Large-Dataset Optimization